0.a initilization list

What?

A way to initialize members BEFORE the constructor body runs

Syntax

C++ **initializer list**

```cpp
class A {
    int x, y;

public:
    A(int a, int b) : x(a), y(b) {
        // constructor body
    }
};
```

Why is better?

- why not like this

C++ **old constructor**

```cpp
A(int a, int b) {
    x = a; // assignment
    y = b;
}
```

Initializer list → direct initialization
Inside body → assignment after default initialization

Why Matters?

Effeciency

C++ **string class**

```cpp
class A {
```

```
    string s;

public:
    A(string str) : s(str) {}  // ✅ direct

    // vs

    A(string str) {
        s = str; // extra work
    }
};
```

1. s default constructed
2. Then assigned → waste

Required in some cases

Const members

C++ **COnst Members**

```
class A {
    const int x;

public:
    A(int a) : x(a) {} // ✅ required
};
```

C++ **fail**

```

A(int a) {
    x = a; // ERROR
}
```

> **Note**
>
> This is will fail because `const` members once initilized cannt be re-initialzed

Reference members

C++ **Reference**

```
class A {
```

```cpp
    int& ref;

public:
    A(int& r) : ref(r) {} // ✅ required
};
```

Base class

C++ **title**

```cpp
class Base {
public:
    Base(int x) {}
};

class Derived : public Base {
public:
    Derived(int x) : Base(x) {} // ✅ required
};
```

Member objects

C++ **Member objects**

```cpp
class B {
public:
    B(int x) {}
};

class A {
    B obj;

public:
    A(int x) : obj(x) {} // ✅ required
};
```

Order of initialization

C++ **Order**

```cpp
class A {
    int x;
    int y;

public:
    A(int a, int b) : y(b), x(a) {}
```

```
7 };
```

- Actual order:
  - x initialized first
  - then y
- Order depends on declaration order, NOT initialization list order